DIGITAL LIBRARY SEARCH AND BOOK RECOMMENDATION

Python Project Documentation

Course: Data Structures and Algorithms

Project Type: Individual PBL Project

Language: Python 3

Interface: Command-line menu

1. Introduction

This educational prototype lets users search a sample book catalogue, browse categories, explore related books, and generate reading-time-aware recommendations. The included data is illustrative and is not a production-scale catalogue.

2. Objectives

• Search by title, author, category, and keyword.
• Represent related books using a graph adjacency list.
• Demonstrate BFS, DFS, Dijkstra, and Floyd–Warshall.
• Compare title/keyword token sequences using LCS.
• Select books under a reading-time limit using 0/1 Knapsack.
• Allocate recommendation slots across categories using DP.

3. Dataset and Graph Model

Each book has an ID, title, author, category, keywords, and estimated reading time. Two books are connected when they share a category or keyword. Positive edge weights indicate simplified relationship costs; lower costs mean closer relationships under this demo rule.

4. Algorithms Applied

Algorithm	Purpose	Complexity
Adjacency list	Store book relationships	O(V + E) storage
BFS / DFS	Discover connected books	O(V + E)
Dijkstra	Lowest-cost path between two books	O((V+E) log V)
Floyd–Warshall	All-pairs shortest paths	$O(V^3)$ time; $O(V^2)$ space
LCS	Compare ordered title/keyword tokens	O(nm) time
0/1 Knapsack	Choose books under a time limit	O(NW) time/space here
Resource allocation DP	Allocate slots across categories with a diversity bonus	Depends on categories, slots and books

5. Running the Project

Open the extracted project folder in VS Code. Open Terminal → New Terminal and run:

```
python main.py
```

On Windows, if needed, use `py main.py`. Run the tests with:

```
python -m unittest discover -s tests -v
```

6. Testing

Automated tests cover search, category browsing, BFS/DFS, Dijkstra path endpoints, Floyd–Warshall diagonal distances, LCS, knapsack capacity, and category allocation. Run the tests on your machine and capture genuine screenshots for the final report.

7. Limitations

The dataset is small and manually curated. Reading times and utility values are estimates. LCS is a simplified similarity feature; recommendations have not been validated on a large real-world dataset. The project has no live catalogue, database, user-history learning, or GUI.

8. Future Scope

Possible extensions include importing a larger dataset, storing preferences in SQLite, adding a GUI/web interface, and evaluating recommendation quality against a documented baseline.

9. Conclusion

The project demonstrates how graph traversal, shortest-path algorithms, LCS, knapsack, and resource-allocation DP can support a small digital library prototype.